Forecasting Indian Floods
From Rain to River

Arijit Vishwakarma (240175)
IIT Kanpur

November 13, 2026

Abstract

AI weather models such as FourCastNet and Pangu-Weather can make a global forecast in seconds, but how well they predict extreme rain over India is not well known. This report tests them on five major Indian floods (Mumbai 2005 and 2024, Delhi 2023, Chennai 2015 and Idukki 2018) and follows the rain down to the river with a physics weather model (WRF), an LSTM and a physics hydrology model (WRF-Hydro). Started about two days ahead, FourCastNet caught the timing of the rain but predicted 85–98% too little for four floods and 49% too little for the three-day Idukki event. On the same 0.25° grid, ERA5, the data it was trained on, already had 31–85% less rain than the IMD gauge-based grid, while FourCastNet stayed within 4–20% of ERA5 in three of four cases, so most of the error is inherited from the training data. Pangu-Weather does not forecast rain, and its pressure and humidity warning signs were not related to flood size ($|r| \leq 0.61$, $p \geq 0.27$, $n = 5$). WRF at 3 km produced local rain peaks above 500 mm for Mumbai 2005, the right order of size, but on the Western Ghats instead of the city; turning off the cumulus scheme on the 9 km grid cut the location error by 62% (221 to 84 km). For Kerala 2018, WRF got the regional three-day rain within 18% of IMD. One LSTM trained on 202 CAMELS-IND basins reached a median test NSE of 0.39 and beat single-basin models in the same 11 basins (0.32 against 0.21). Uncalibrated WRF-Hydro reproduced the 2018 Kallada flood peak within 2–16% of the observed 937 m^3/s, and IMD rain raised its NSE from 0.04 to 0.38 compared with ERA5 rain.

On the computing side, a free T4 GPU ran the AI and LSTM models 4–20 times faster than a laptop. On a 24-core server, WRF ran 1.8 times faster than on the laptop, but only on the 8 performance cores; using all cores was always slower. Distributed PyTorch (DDP) made LSTM training up to 8.4 times faster, and WRF-Hydro over Kerala ran 12.5 times faster on 128 MPI processes across 8 machines (5314 s to 426 s). Overall, these models capture the large-scale monsoon well, but local rainfall amounts, and the data the models learn from, remain the main limit for flood forecasting in India.

Contents

1 Introduction **2**

2 Data **3**
2.1 ERA5 3
2.2 IMD Rainfall 4
2.3 Flood Events 4
2.4 River Data 4
2.5 Maps of the Ground 5

3 AI Weather Models Used **5**
3.1 FourCastNet 5
3.2 Pangu-Weather 5

3.3 Running the Models . . . 6
3.4 How the Models Were Tested . . . 6

4 Results **6**
4.1 FourCastNet: Predicted vs Observed Rain . . . 6
4.2 FourCastNet: Where Does the Error Come From? . . . 8
4.3 Pangu-Weather: Warning Signs . . . 8

5 Discussion **10**

6 Limitations **10**

7 High-Resolution Downscaling with WRF **10**
7.1 Why WRF . . . 10
7.2 Model Setup . . . 11
7.3 Experiments . . . 11
7.4 Results: Rain Intensity and Grid Size . . . 11
7.5 Comparison with the IMD Grid . . . 13
7.6 Finding the Cause of the Location Error . . . 14
7.7 A Second Event: the Kerala Floods of August 2018 . . . 14
7.8 What the WRF Experiments Show . . . 16

8 Streamflow Prediction with NeuralHydrology **16**
8.1 Earlier Single-Basin Experiments . . . 17
8.2 One Model for All CAMELS-IND Basins . . . 17

9 Flood Prediction with WRF-Hydro **20**

10 Computational Performance **21**
10.1 Summary for All Models . . . 22
10.2 FourCastNet and Pangu-Weather . . . 22
10.3 WRF . . . 23
10.4 NeuralHydrology . . . 24
10.5 Distributed PyTorch (DDP) . . . 24
10.6 WRF-Hydro on Eight Lab Machines . . . 25
10.7 Main Points . . . 27

11 Conclusion **27**

Code and Implementation **28**

1 Introduction

Weather forecasts are normally made with physics models that run on supercomputers. In the last few years, AI models have appeared that learn the weather from decades of past data and can forecast much faster. In this project I followed Indian floods from the rain to the river, using five models along the way: two AI weather models, one physics weather model, and two river-flow models (one AI and one physics).

- **FourCastNet** (NVIDIA, 2022) is an AI weather model trained on ERA5 data from 1979 to 2015. It covers the whole globe on a grid of about 25 km and moves the forecast forward in 6-hour steps. It forecasts temperature, humidity, wind and pressure, and has a separate part that forecasts rainfall.

- **Pangu-Weather** (Huawei, 2023) is another AI weather model, trained on ERA5 from 1979 to 2017. It sees the air in 3D, at the surface and at 13 pressure levels above it. It forecasts temperature, humidity, wind and pressure, but *not* rainfall.
- **WRF** (Weather Research and Forecasting model, NCAR) is a physics model. Instead of learning from past data, it solves the equations of air, heat and water step by step. I ran it in nested boxes of 27, 9 and 3 km over each flood, starting from ERA5 (Section 7).
- **LSTM** is a neural network with memory. With the NeuralHydrology library I trained it to read the weather of the past 365 days for a river basin and predict the daily river flow, first for single basins and then as one model for many CAMELS-IND basins (Section 8).
- **WRF-Hydro** (NCAR) is a physics model of how rain becomes river flow. It moves the water through the soil, over the land and along the river channels. I used it for the 2018 Kerala flood on the Kallada river at Pattazhy (Section 9).

I did not train the two AI weather models. I used the trained versions published by their authors and only ran them.

2 Data

This project used four kinds of data: past weather, observed rain, river records and maps of the ground. Table 1 lists every dataset, and the parts below explain each one.

Table 1: All datasets used in this project

Dataset	What it contains	Grid / time step	Used for
ERA5 (ECMWF)	Past global weather (reanalysis)	0.25°, hourly	Starting weather for FourCastNet and Pangu-Weather; starting and edge weather for WRF; weather input for WRF-Hydro
IMD gridded rain	Daily rain from about 7,000 gauges	0.25°, daily	Checking FourCastNet and WRF; rain input for the second WRF-Hydro run
IMD station and event reports	Rain observed on each flood day	Single points	Size of each flood (Table 2)
CAMELS-IND (v2.2)	River flow, weather and basin properties for Indian rivers	Daily, per basin	Training and testing the LSTM; checking WRF-Hydro at Pattazhy
CAMELS US	The same kind of data for US rivers	Daily, per basin	Checking the LSTM setup on one basin
NCEP FNL	Global weather analysis	Global, 6-hourly	LSTM test with a different weather input
Copernicus DEM	Height of the ground	30 m	River network for WRF-Hydro
WRF static maps	Ground height, land use and soil	Fixed maps	Land grids for WRF and WRF-Hydro

2.1 ERA5

ERA5 is ECMWF's record of past weather for the whole globe, from 1940 to today, on a 0.25° grid (about 25 km) for every hour (Hersbach et al., 2020). It is made by combining a weather model with millions of past observations, so it is the best estimate of what the weather actually was. Both AI models were trained on ERA5. I downloaded it from the Copernicus Climate Data Store (CDS) and used it in four ways:

- as the starting weather (one snapshot) for every FourCastNet and Pangu-Weather forecast;
- as the starting weather, and the weather at the edges of the region, for WRF (every 6 hours);
- as the hourly weather input (rain, temperature, humidity, wind, radiation and pressure) for WRF-Hydro;
- as a second rain record to compare with FourCastNet, because it is the rain FourCastNet learned from.

2.2 IMD Rainfall

IMD gridded rain. The India Meteorological Department makes a daily rain map of India on a 0.25° grid from about 7,000 rain gauges (Pai et al., 2014). It is on the same grid as ERA5 and the AI models, so it is the fairest record to compare them with. IMD's rain day runs from 08:30 to 08:30 IST. I read it with the `imdlib` Python package. It was used to check FourCastNet and WRF, and as the rain input for the second WRF-Hydro run.

Station and event reports. For the size of each flood I used the rain reported by IMD gauges (Santacruz for Mumbai, Safdarjung for Delhi) and IMD event reports for Chennai and Idukki (Table 2).

For each city I used the nearest grid cell. A gauge measures one spot, while a grid cell averages about 25 km, so a gauge can show much more rain in a local storm.

2.3 Flood Events

I chose five well-known floods from different parts of India (Table 2). The start date is the day the AI forecast began, about two days before the flood.

Table 2: Flood events used in this study

Case	City	Start date	Observed rainfall	Notes
Mumbai_2005	Mumbai	24 Jul 2005	944 mm / 24h	IMD Santacruz; Mumbai's wettest day on record
Mumbai_2024	Mumbai	6 Jul 2024	300 mm / 6h	IMD red alert, 01:00–07:00 IST
Delhi_2023	Delhi	7 Jul 2023	153 mm / 24h	IMD Safdarjung
Chennai_2015	Chennai	29 Nov 2015	286 mm / 24h	City average
Idukki_2018	Idukki, Kerala	13 Aug 2018	588 mm / 3 days	Peak of the 2018 Kerala floods

2.4 River Data

CAMELS-IND (version 2.2) is a public dataset of Indian river basins. For each basin it gives the daily river flow at a gauge, 18 daily weather variables averaged over the basin (rain from the IMD grid, temperature, radiation, wind, humidity, evaporation and soil moisture), and fixed properties such as size, height, slope, land use, soil, geology and dams. I used the 242 basins with enough flow records and kept 202 after cleaning (Section 8.2). The flow at the Pattazhy gauge (15022) on the Kallada river was also used to check WRF-Hydro.

CAMELS US is the same kind of dataset for US rivers. I used one basin (01022500) only to check that my LSTM setup gave the same score as the NeuralHydrology tutorial.

NCEP FNL (NSF NCAR GDEX, dataset `d083002`) is a global weather analysis every 6 hours. I tested it as a different weather input for the LSTM.

2.5 Maps of the Ground

WRF needs maps of ground height, land use and soil to work out how mountains and land change the weather. I used WRF's standard static maps (the WPS geographical data). WRF-Hydro used the same maps for its 1 km land grid. Its 250 m river grid was made from the Copernicus 30 m DEM (a map of ground height), together with the official CAMELS-IND basin outline.

3 AI Weather Models Used

Both AI models learn how the weather changes from about 40 years of ERA5. Given the weather at one moment, they predict the weather a few hours later. To forecast further ahead, they feed their own prediction back in as the next input, again and again. They do not use the equations of physics; everything they know comes from the data. Table 3 compares the two models.

3.1 FourCastNet

FourCastNet (Fourier Forecasting Neural Network) was made by NVIDIA and university partners in 2022 (Pathak et al., 2022).

How it works. The globe is cut into small square patches, the way a picture is cut into tiles. The network then mixes information between the patches using a Fourier transform, which breaks the weather pattern into waves of different sizes. This step is called an Adaptive Fourier Neural Operator (AFNO). It lets every part of the globe affect every other part, and it stays fast on a GPU even on a 25 km grid.

What it forecasts. Wind, temperature and pressure near the surface; wind, temperature, humidity and height at several levels in the air (such as 850 and 500 hPa); and the total water vapour in the air. In the original paper this is 20 variables. Each step moves the forecast 6 hours ahead.

Rain. Rain is hard to learn because it is patchy and changes quickly. So FourCastNet has a second network that takes the main forecast and works out the rain that fell in each 6 hours. This is why FourCastNet can be checked directly against rain gauges.

Training. ERA5 from 1979 to 2015, with 2016–2017 for tuning and 2018 for testing. Once trained, a week-long global forecast takes only seconds on one GPU. I used version 0.1 from the `ai-models` tool, which downloads both networks (the main one and the rain one).

3.2 Pangu-Weather

Pangu-Weather was made by Huawei Cloud and published in *Nature* in 2023 (Bi et al., 2023).

How it works. It treats the atmosphere as a 3D block: latitude, longitude and height. Its network, the 3D Earth-Specific Transformer, looks at small 3D windows of air and learns how each one affects its neighbours. "Earth-specific" means it knows where on the globe each window is, because the weather near the equator behaves differently from the weather near the poles.

What it forecasts. Four surface variables (sea-level pressure, 2 m temperature and the two parts of the 10 m wind) and five variables in the air (height, temperature, humidity and the two parts of the wind) at 13 pressure levels from 1000 to 50 hPa. It does *not* forecast rain.

Four models in one. Pangu-Weather has four separately trained models that jump 1, 3, 6 and 24 hours ahead. To reach a lead time it uses the biggest jumps first; for example, 54 hours is two 24-hour jumps and one 6-hour jump. Fewer jumps means less error builds up along the way. The `ai-models` version I used has the 6-hour and 24-hour models.

Training. ERA5 from 1979 to 2017, with 2019 for tuning and 2018 for testing. In the paper it was more accurate than ECMWF's main physics forecast for most variables, and far faster.

Table 3: The two AI weather models side by side

	FourCastNet	**Pangu-Weather**
Made by	NVIDIA and partners (2022)	Huawei Cloud (2023)
Network	AFNO: mixes patches of the globe with a Fourier transform	3D Earth-Specific Transformer
Training data	ERA5, 1979–2015	ERA5, 1979–2017
Grid	0.25° (about 25 km), whole globe	0.25° (about 25 km), whole globe
Levels in the air	A few (such as 850 and 500 hPa)	13 (1000 to 50 hPa)
Time step	6 hours	6 and 24 hours (in `ai-models`)
Rain forecast	Yes, from a separate network	No
How I tested it	Rain against gauges and the IMD grid	Pressure and humidity warning signs

3.3 Running the Models

I used ECMWF's `ai-models` tool, which downloads the trained models, gets the ERA5 starting data from CDS and writes the forecast. Everything ran on a free Google Colab GPU (NVIDIA T4). Pangu-Weather did not fit in the T4's memory at first, so I kept its 6-hour model on the GPU and moved the 24-hour model, which is used only every fourth step, to the CPU. This does not change the forecast. Each forecast started at 00:00 UTC about two days before the flood, so the models had not seen the flood day (Table 2).

3.4 How the Models Were Tested

FourCastNet forecasts rain, so I compared its rain directly with the observations. Pangu-Weather does not, so I checked two warning signs of heavy rain instead: falling sea-level pressure and rising humidity at about 1.5 km (850 hPa).

Reading FourCastNet's rain. FourCastNet saves rain as a running total since the start of the forecast. So the rain between two times is the total at the end minus the total at the start. I measured every forecast over exactly the same hours as its observation. IMD's rain day (08:30–08:30 IST) is 03:00–03:00 UTC; for Idukki I used three IMD days (15–17 August).

Pangu-Weather warning signs. At each city I compared the average pressure and humidity of the last 24 hours of the forecast with the first 24 hours. Using full days removes the normal twice-a-day rise and fall of pressure. A useful warning sign should be stronger before bigger floods.

4 Results

4.1 FourCastNet: Predicted vs Observed Rain

Table 4: FourCastNet predicted vs observed rainfall over the same time window

Case	**Observed (mm)**	**Predicted (mm)**	**Error (mm)**	**Error (%)**
Delhi_2023	153	10.8	−142.2	−92.9
Chennai_2015	286	44.3	−241.7	−84.5
Mumbai_2024	300	6.3	−293.7	−97.9
Idukki_2018	588	302.8	−285.2	−48.5
Mumbai_2005	944	59.8	−884.2	−93.7

FourCastNet predicted far too little rain for **every** flood. Idukki, a long three-day monsoon spell, was the best case (−49%). The short, intense floods were the worst: 85–98% too little.

The model gave steady, light rain of about 5–9 mm every 6 hours, with no burst. For Mumbai 2024, it gave only 6.3 mm in the six hours when 300 mm fell, and its wettest hours came about 10 hours too late.

Bigger floods are not missed more. The error did not depend on flood size ($r = -0.19$, $p = 0.76$, $n = 5$, Figure 1). The model misses all of them by a lot.

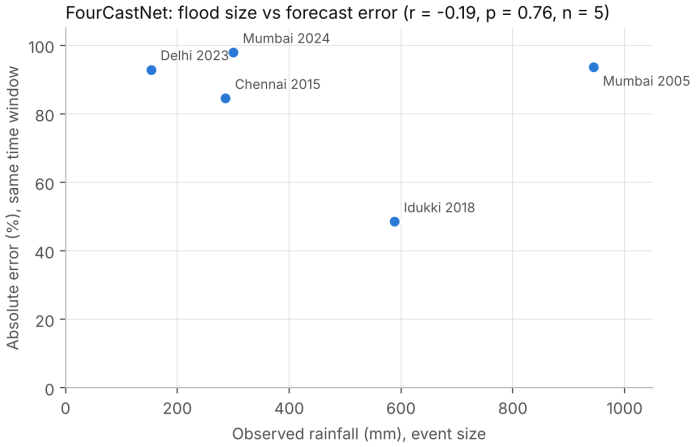


Figure 1: FourCastNet: observed rainfall (flood size) vs error.

Starting earlier does not change much. I started the Mumbai 2005 forecast 1, 2, 3 and 5 days before the flood (Table 5, Figure 2). The flood-day rain stayed between 41 and 67 mm in every case, so the problem is not forecasting too far ahead.

Table 5: Mumbai 2005: flood-day rainfall vs forecast start (observed: 944 mm)

Started (days before)	Predicted (mm)	Error (%)
1	40.8	−95.7
2	59.8	−93.7
3	66.8	−92.9
5	53.6	−94.3

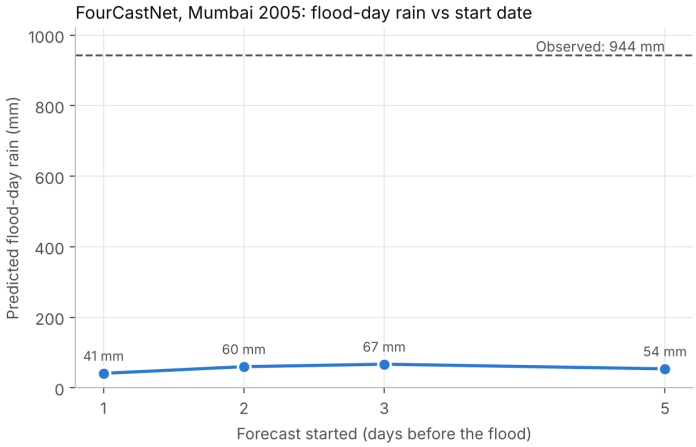


Figure 2: FourCastNet, Mumbai 2005: predicted flood-day rain vs forecast start.

4.2 FourCastNet: Where Does the Error Come From?

To find out where the rain is lost, I compared four sources for the same window (Table 6, Figure 3).

Table 6: Rainfall from four sources over the same window (mm)

Case	Gauge	IMD grid	ERA5	FourCastNet	IMD grid vs gauge	ERA5 vs IMD grid	FourCastNet vs ERA5
Mumbai_2005	944	423.4	74.3	59.8	−55%	−82%	−20%
Delhi_2023	153	61.4	34.9	10.8	−60%	−43%	−69%
Chennai_2015	286	329.3	50.6	44.3	+15%	−85%	−12%
Idukki_2018	588	455.5	314.9	302.8	−23%	−31%	−4%
Mumbai_2024	300	–	2.5	6.3	–	–	+3.8 mm

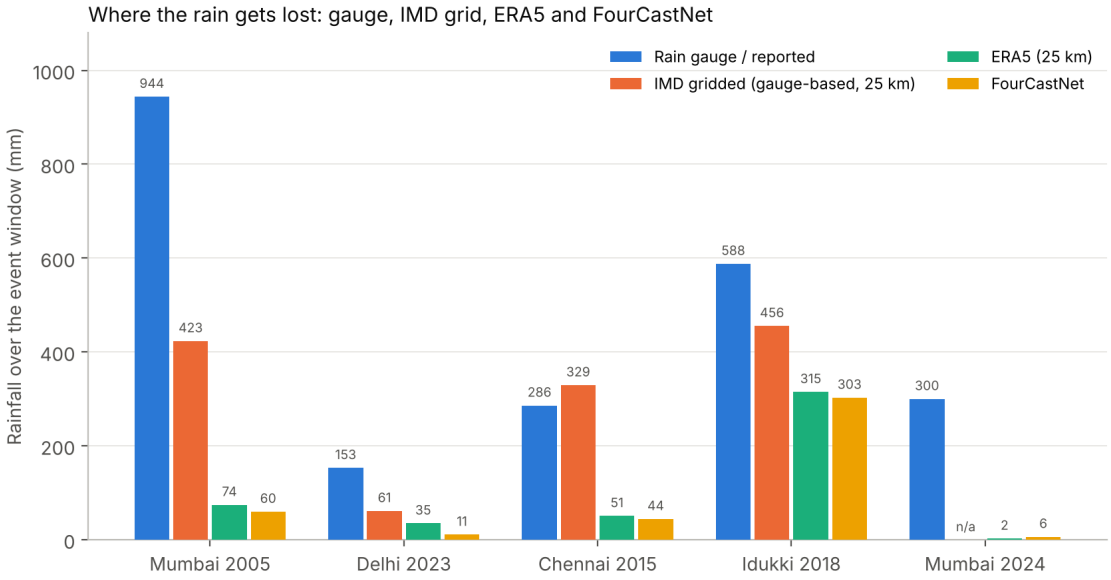


Figure 3: Rainfall over each event window from the gauge, the IMD grid, ERA5 and FourCastNet.

- **Gauge to IMD grid:** averaging over 25 km loses some rain (23–60%), except for Chennai (+15%).
- **IMD grid to ERA5: the biggest loss.** Even on the same grid, ERA5 has 31–85% less rain than the measured IMD grid.
- **ERA5 to FourCastNet:** in three cases FourCastNet is within 4–20% of ERA5. Only Delhi is clearly worse (−69%).

For Mumbai 2005, for example, the rain goes from 944 mm at the gauge to 423 mm in the IMD grid, 74 mm in ERA5 and 60 mm in FourCastNet. So **most of the rain is already missing in ERA5, the data FourCastNet learned from**. Against the IMD grid, FourCastNet was 82–87% too low for Mumbai 2005, Delhi and Chennai, and 34% too low for Idukki.

4.3 Pangu-Weather: Warning Signs

Pangu-Weather's forecasts looked realistic: each region had the right pressure level, and pressure rose and fell twice a day as it does in the tropics (Figure 4). But the warning signs did not

match the floods (Table 7, Figure 5).

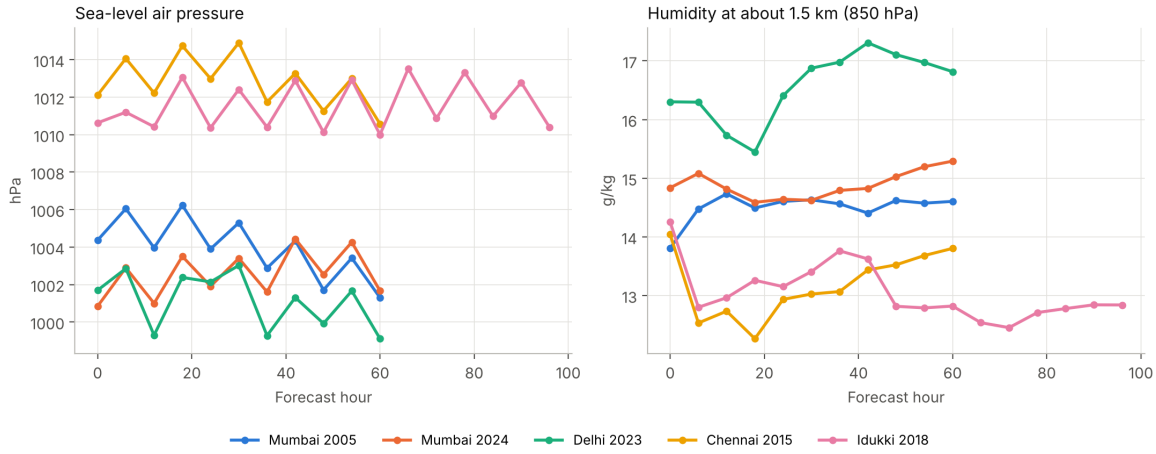


Figure 4: Pangu-Weather forecast of sea-level pressure (left) and 850 hPa humidity (right) at each city.

Table 7: Pangu-Weather warning signs (last 24 h average minus first 24 h average)

Case	Observed (mm)	ΔP (hPa)	Δq_{850} (g/kg)	Signal
Delhi_2023	153	−1.06	+1.11	Both present; largest moisture rise
Chennai_2015	286	−1.26	+0.72	Both present
Mumbai_2024	300	+1.17	+0.25	Weak moisture only
Idukki_2018	588	+0.35	−0.53	None (drying)
Mumbai_2005	944	−2.46	+0.17	Pressure only

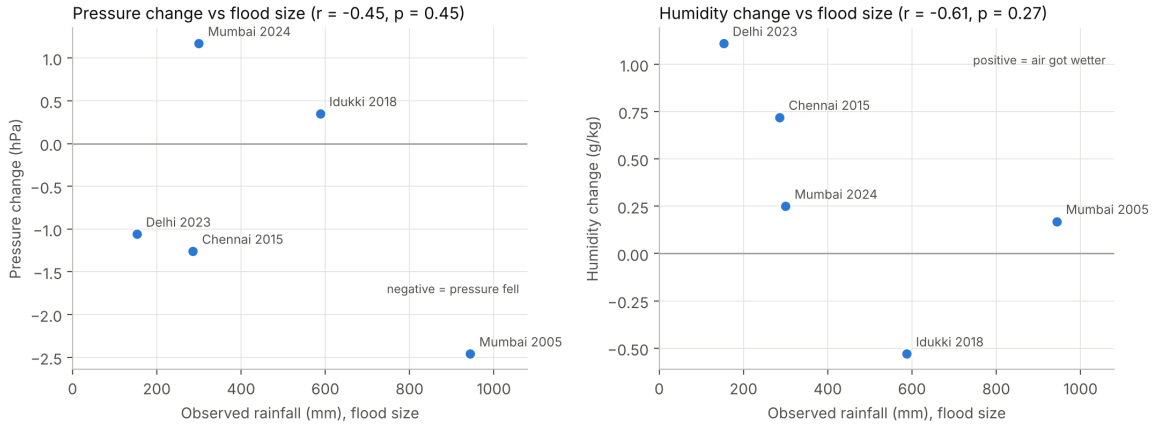


Figure 5: Pangu-Weather: pressure change (left) and humidity change (right) vs observed rainfall.

The smallest flood (Delhi) had the biggest humidity rise, and the two biggest floods had little (Mumbai 2005) or even falling (Idukki) humidity. The pressure fall was largest for Mumbai 2005, but not for the other big floods. Neither link was statistically significant (pressure: $r = -0.45$, $p = 0.45$; humidity: $r = -0.61$, $p = 0.27$).

5 Discussion

Both models get the big picture right. FourCastNet expected more rain in the days before each flood, and Pangu-Weather gave realistic pressure and humidity patterns.

But FourCastNet badly underestimates heavy rain, mainly because of its training data. It copies ERA5 quite closely, and ERA5 itself has far too little monsoon rain. A model trained on ERA5 cannot do better than ERA5. Training on average errors also pushes AI models towards smooth, moderate values instead of sharp peaks. Idukki was the best case because it was a long, widespread event that ERA5 also captured reasonably well.

Pangu-Weather's warning signs do not show how big a flood will be. Heavy rain needs moisture *and* something that lifts the air, such as mountains or converging winds. Pressure and humidity at a single point cannot show this, which is why the Kerala floods, driven by monsoon winds hitting the Western Ghats, showed no warning at all.

Overall: these AI models are useful for the large-scale weather, but they should not be used alone to judge how intense flood rain will be. With only five events, this is a clear pattern rather than a proven result.

6 Limitations

1. **Few events.** Five floods are too few for reliable statistics.
2. **Different observations.** The reported values are single gauges, a city average and a basin total. The IMD grid comparison helps but does not remove this problem.
3. **Time windows.** The three Idukki days are my choice based on reports of 15–17 August, and the IMD grid could not be used for the six-hour Mumbai 2024 event.
4. **Training-period overlap.** Mumbai 2005, Chennai 2015 and Idukki 2018 fall inside the years the models were trained on. Delhi 2023 and Mumbai 2024 come after, and were still badly missed.
5. **Simple warning signs.** I used only two variables at one grid point for Pangu-Weather.
6. **Speed tests.** Each timing was measured once. The DDP test used synthetic data and the Pangu-Weather timing used random input of the real size; the laptop and server used different PyTorch versions, and FourCastNet was timed only on the GPU.
7. **WRF set-up.** Six WRF runs for two events, one run per setting, with no data assimilation or ensemble. The cumulus result is strong evidence for the Mumbai storm, not a general rule.
8. **No retraining.** I used the published models; retraining them needs far more GPU time than free resources give.

7 High-Resolution Downscaling with WRF

7.1 Why WRF

FourCastNet and Pangu-Weather work on a fixed 25 km grid. The Mumbai cloudburst of 26 July 2005 was only 5–10 km wide, so on a 25 km grid its rain gets spread over a big box and the peak is lost. The AI models cannot be run on a finer grid.

So I used WRF (version 4.8.0), a physics-based weather model that can run on a small region at 3 km. I gave it ERA5 as the starting weather and as the weather at the edges of the region, and WRF worked out the small-scale detail itself, using detailed maps of the mountains and land.

7.2 Model Setup

I ran WRF on my laptop (16 processes) and later on the CSE server. I used three boxes, one inside the other: 27 km, 9 km and 3 km (Table 8).

Table 8: WRF domains (centred at 19°N, 76°E)

Domain	Grid size	Grid points	Area covered
d01	27 km	100 × 100	about 63–89°E, 7–31°N (most of India)
d02	9 km	151 × 151	about 67–80°E, 13–25°N (western India)
d03	3 km	151 × 151	about 70.7–75.0°E, 17.0–21.0°N (Mumbai and Ghats)

ERA5 data came every 6 hours. The *cumulus scheme* is a simple formula WRF uses to guess rain from storm clouds that are too small for the grid. I kept it on for the 27 and 9 km boxes and off for the 3 km box, which can show storms directly. I added up the rain over the IMD rain day (08:30 to 08:30 IST, 26–27 July), the same as for FourCastNet.

7.3 Experiments

I made five runs (Table 9). Runs A–C test the start time and the physics settings. Runs D and E are copies of run B with one change each; I added them later to find out why the rain was in the wrong place.

Table 9: WRF experiments

Run	Start (UTC)	Length	Cloud physics / cumulus scheme	Change from B
A	25 Jul 00:00	60 h	WSM6 / new Tiedtke (27, 9 km)	starts 1 day earlier
B	26 Jul 00:00	36 h	WSM6 / new Tiedtke (27, 9 km)	–
C	25 Jul 00:00	60 h	Thompson / Kain–Fritsch (27, 9 km)	physics, earlier start
D	26 Jul 00:00	36 h	as B	spectral nudging on 27 km
E	26 Jul 00:00	36 h	WSM6 / new Tiedtke (27 km only)	no cumulus on 9 km

Runs A–C ran on the laptop and runs D–E on the CSE server.

7.4 Results: Rain Intensity and Grid Size

Table 10 gives WRF's rain at the Santacruz gauge, the largest value within 25 and 50 km of it, and the heaviest rain anywhere in the 3 km box. (For runs D and E I kept only the rain on the IMD grid, because of the server's disk limit.)

Table 10: Flood-day rain (mm) from WRF compared with observations (Santacruz gauge 944 mm, IMD grid cell 423 mm, ERA5 74 mm, FourCastNet 60 mm)

Run	Grid	At gauge	Max <25 km	Max <50 km	Regional peak	Peak location
A	27 km	10.1	11.8	31.3	280.0	219 km S
A	9 km	11.5	16.4	52.7	453.7	128 km SE
A	3 km	17.6	23.3	78.4	572.5	210 km SE
B	27 km	13.2	13.2	49.9	237.7	194 km S
B	9 km	5.3	23.9	96.4	417.4	103 km E
B	3 km	4.5	32.6	124.7	518.2	99 km E
C	27 km	7.2	7.2	14.2	132.5	244 km S
C	9 km	3.6	9.9	26.2	156.8	235 km S
C	3 km	4.4	20.7	38.8	112.0	148 km SE

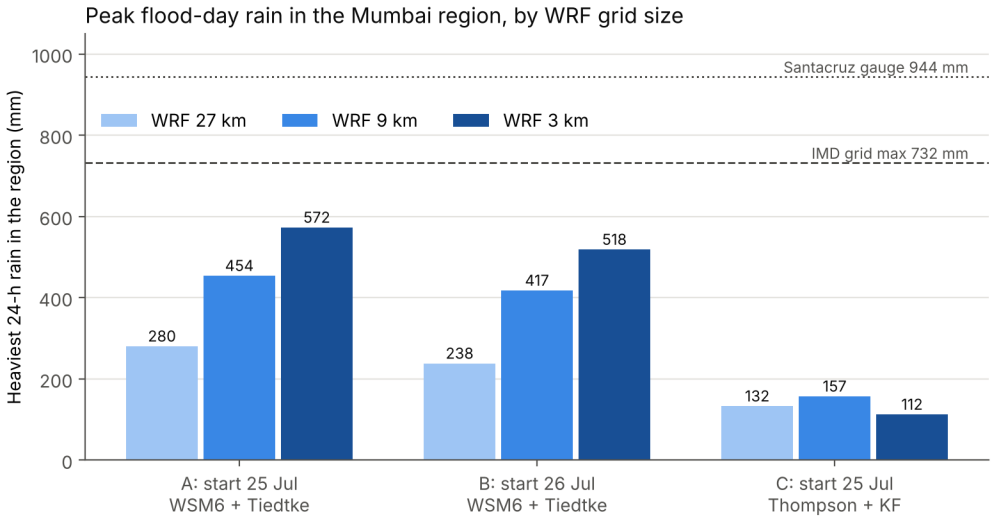


Figure 6: Heaviest flood-day rain in the Mumbai region for each run and grid size. With the WSM6 setup, going from 27 to 3 km doubles the peak (A: 280 → 573 mm).

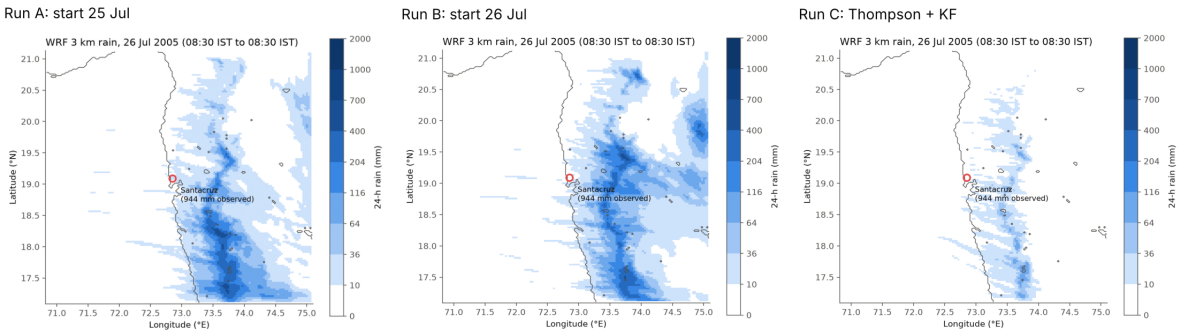


Figure 7: WRF 3 km flood-day rain for runs A, B and C. The red circle marks Santacruz.

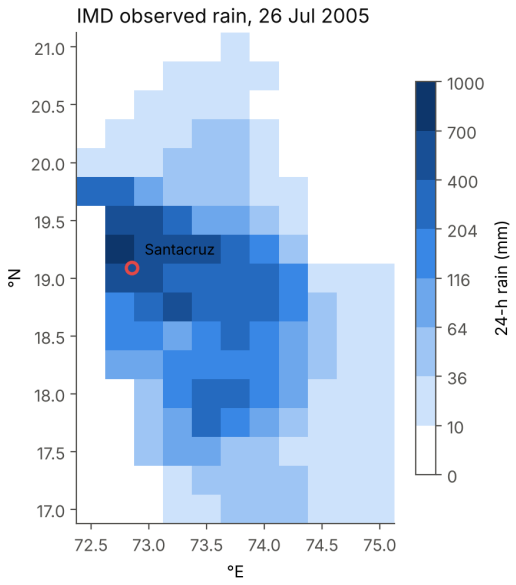


Figure 8: IMD gauge-based rain for the same day on its 0.25° grid. The heaviest rain (up to 732 mm) is at Mumbai and Thane, with 150–440 mm also over the Ghats to the east and south.

What I found:

1. **A finer grid gives much heavier rain.** The peak went from 280 mm at 27 km to 573 mm at 3 km in run A (238 to 518 mm in run B). FourCastNet's largest value at Mumbai was only 60 mm.

2. **But the rain fell in the wrong place.** In every run the heavy rain fell on the Western Ghats, 100–240 km away, and not over Mumbai city. At Santacruz WRF gave only 4–18 mm.

3. **The physics settings matter as much as the grid.** Changing only the cloud and cumulus schemes (run C) cut the peak five times, from 573 to 112 mm.

7.5 Comparison with the IMD Grid

Checking a single gauge is unfair if the rain is shifted a little. So I averaged WRF's 3 km rain into the same 25 km boxes as the IMD grid and compared the whole region (165 land boxes). CSI tells how well the heavy-rain areas overlap (1 = perfect, 0 = no overlap).

Table 11: WRF (3 km, averaged to 0.25°) against IMD gridded rain over the Mumbai region

	Mean (mm)	Max (mm)	Bias (%)	Pattern corr.	CSI ≥64.5	CSI ≥115.6	CSI ≥204.5
IMD (observed)	91	732	–	–	–	–	–
Run A	31	272	−66	0.17	0.26	0.16	0.00
Run B	36	272	−60	0.27	0.33	0.17	0.00
Run C	6	60	−94	0.29	0.00	0.00	0.00
Run D (nudging)	33	257	−64	0.27	0.32	0.15	0.00
Run E (no 9 km cumulus)	36	236	−60	**0.39**	**0.42**	**0.20**	**0.12**

Of runs A–C, run B was the best, with about 40% of the observed rain. None of them matched the very heavy boxes (≥204.5 mm) near Mumbai. Also, averaging to 25 km brought WRF's 518–573 mm peaks down to 272 mm, the same smoothing that hurts the AI models.

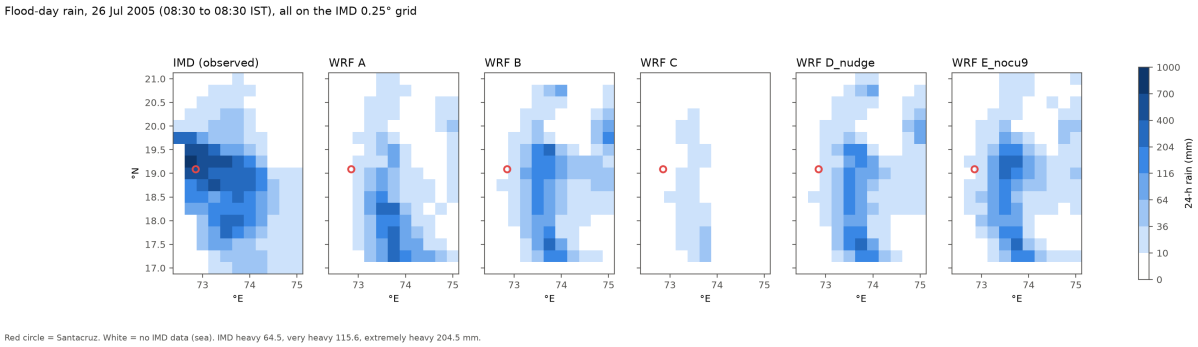


Figure 9: IMD and WRF runs A–E on the same 0.25° grid. The red circle marks Santacruz.

7.6 Finding the Cause of the Location Error

Table 12 shows where each run put its heaviest rain, compared with the IMD peak at Mumbai and Thane.

Table 12: Position of the heaviest rain box on the 0.25° grid

	Peak box	Distance from IMD peak
IMD (observed)	19.25°N 72.75°E	–
Run A	18.25°N 73.50°E	136 km
Runs B, C, D	17.50°N 73.75°E	221 km
Run E	19.00°N 73.50°E	**84 km**

Run D: is the large-scale weather wrong? Nudging keeps WRF's large-scale winds close to ERA5. Run D came out almost the same as run B, so the large-scale weather was already right.

Run E: is the cumulus scheme to blame? I switched the cumulus scheme off on the 9 km box. The rain moved north to Mumbai's latitude, the error fell from 221 to 84 km, the correlation rose from 0.27 to 0.39, and some very heavy rain boxes were matched for the first time. So the cumulus scheme caused much of the error.

Run E is still not perfect: the rain is 60% too low (highest box 236 mm against 732 mm) and it falls 50–80 km inland on the Ghats instead of on the coast.

7.7 A Second Event: the Kerala Floods of August 2018

The Mumbai storm sat over the coast, which is very hard for any model. So I also ran the 2018 Kerala floods (the Idukki event), where the rain is caused mainly by mountains. I moved the same three boxes to Kerala and used run E's settings. Like FourCastNet, the run started on 13 August 2018 and lasted 102 hours, and I added up the rain over 15–17 August.

Table 13: WRF against the IMD grid for the two events (3 km rain averaged to 0.25°)

	Kerala 2018 (3 days)	Mumbai 2005, run E (1 day)
IMD mean / max (mm)	144 / 696	91 / 732
WRF mean / max (mm)	118 / 1054	36 / 236
Bias of the regional mean	−18%	−60%
Pattern correlation	0.41	0.39
CSI, heavy rain (per day)	0.43–0.45	0.42
Distance between heaviest boxes	197 km	84 km

Table 14: Three-day rain at Idukki (mm), 14–17 August 2018

Source	Rain (mm)
IMD grid cell (observed)	455.5
ERA5	314.9
FourCastNet	302.8
WRF, averaged over the same cell	250.5
WRF, largest value within 25 km	708

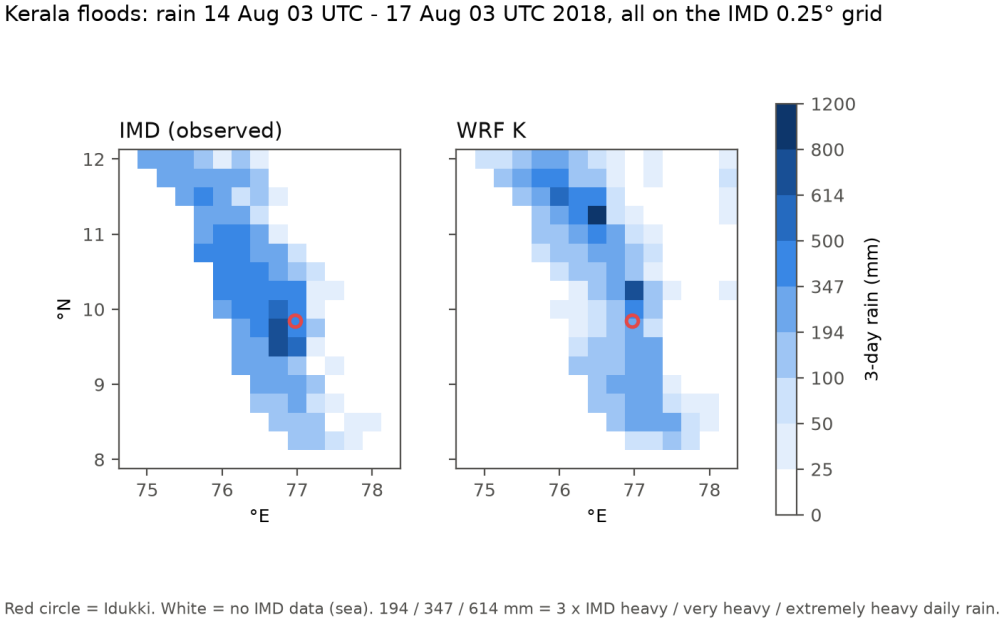


Figure 10: Three-day rain for the Kerala floods from IMD and WRF on the same 0.25° grid. The red circle marks Idukki.

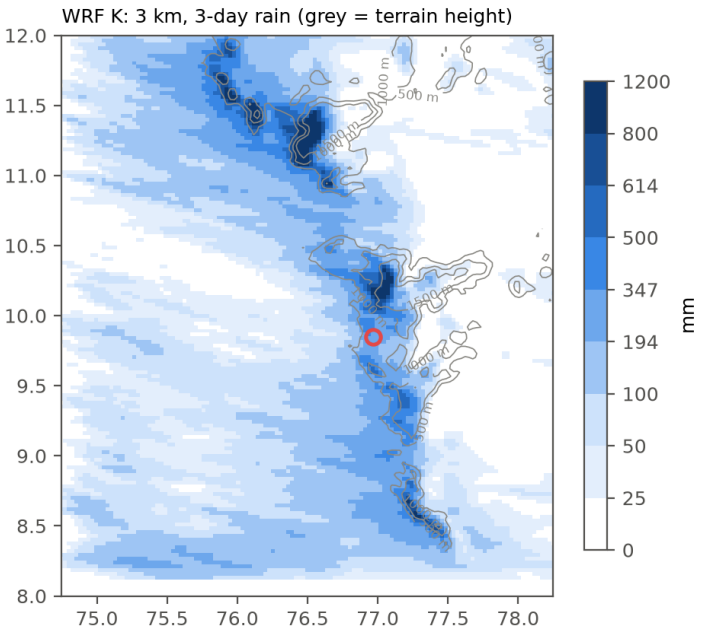


Figure 11: WRF three-day rain at 3 km with terrain heights (grey lines). The heaviest rain lies on the highest mountains.

What I found:

1. **The total rain was much better than for Mumbai**: only 18% too low (60% for Mumbai), and the rain band lay along the Ghats as observed.
2. **But the rain was piled onto the mountain tops**, up to 2023 mm at one point, with too little on the lower slopes.
3. **At Idukki itself WRF was not better**: 251 mm against 456 mm observed, less than ERA5 (315 mm) and FourCastNet (303 mm), even though 708 mm fell within 25 km.

7.8 What the WRF Experiments Show

A 3 km grid does give heavy rain of the right size (500–570 mm), so the coarse grid is one reason the AI models miss extreme rain. But a finer grid alone is not enough. WRF put the Mumbai rain on the Ghats, and the result changed a lot with the physics settings. Switching off the cumulus scheme on the 9 km box fixed much of the location error (221 to 84 km). For Kerala, WRF got the regional total close (−18%), but in both events it put the heaviest rain on the mountain tops, so the rain at one town can still be far off. A good next step would be to use an AI model like Pangu-Weather for the large-scale forecast and WRF (with run E's settings) for the detailed rain.

8 Streamflow Prediction with NeuralHydrology

Floods are finally about rivers, so the last part of this work predicts daily river flow. An LSTM (a neural network with memory) reads the weather of the previous 365 days for a catchment and predicts the flow on the last day. Its memory suits rivers, because rain is stored in soil and groundwater for weeks before it reaches the river (Kratzert et al., 2018). All models were built with NeuralHydrology (Kratzert et al., 2022), an open-source PyTorch library in which each experiment is described by one configuration file.

Two scores are used. The **Nash–Sutcliffe efficiency (NSE)** is 1 for a perfect model and 0 for a model that only predicts the average flow; negative values are worse than that (Nash and Sutcliffe, 1970). The **Kling–Gupta efficiency (KGE)** combines timing, the size of the ups and downs, and the total volume; it is also 1 when perfect (Gupta et al., 2009).

8.1 Earlier Single-Basin Experiments

The first experiments trained one model on one basin, with the settings of NeuralHydrology's tutorial (CudaLSTM with 20 hidden units, MSE loss, 50 epochs). Table 15 gives the results.

Table 15: Single-basin experiments (test period scores)

Experiment	Data and periods	NSE	KGE
1. Check on CAMELS US	Basin 01022500, tutorial setup	0.776	–
2. CAMELS-IND basin 3002	18 CAMELS-IND inputs; train 1993–2008, test 2012–2018	0.004	–
3. Basin 3002, three weather inputs (train 2012–2015; test 2017–Oct 2018, wrong values removed)			
A: NCEP FNL only	FNL temperature, humidity, wind, precipitable water, soil moisture	−0.30	0.01
B: FNL + CAMELS-IND rain	As A, plus observed rain	−0.39	0.24
C: CAMELS-IND only	Rain, temperature, humidity, wind	**−0.19**	**0.38**

- **Experiment 1** reproduced the tutorial's reference value (0.777), so the setup was correct.
- **Experiment 2** scored almost zero. The reason was found later: on five days in September 2018 the observed flow is exactly 946.47 mm/day, an impossible value that dominates the score.
- **Experiment 3** tested NCEP FNL global data (NSF NCAR GDEX, dataset `d083002`). FNL has no rainfall before June 2019, so precipitable water and soil moisture were used instead. FNL caught the timing of the monsoon (correlation 0.63) but overestimated flow. CAMELS-IND's own data, with gauge-based IMD rain, gave the best KGE (0.38).

These models were limited by one basin, very little training data (889 samples in Experiment 3) and wrong flow values. The next experiment addresses all three.

8.2 One Model for All CAMELS-IND Basins

LSTMs for river flow work much better when one model learns from many basins at once, with fixed catchment properties as extra inputs, because it can learn how rain becomes flow from hundreds of rivers instead of one. This was tested on CAMELS-IND.

Data. The 242 CAMELS-IND catchments with enough observed flow (version 2.2). The rain input is the IMD 0.25° gauge-based rain averaged over each catchment.

Cleaning the flow data. Before training, three kinds of wrong values were removed automatically: negative flows, the same extreme value repeated on separate days (such as the 946.47 mm/day values of basin 3002), and flow more than 100 mm/day above the rain of the previous week. This removed 527 values in 40 basins. Basins were kept if they had at least three years of flow for training and one year for testing, and if their flow was not larger than their rain (a runoff ratio above 1 usually means a wrong catchment area). **202 basins** were kept.

Inputs. All 18 daily CAMELS-IND weather variables (rain, temperature, radiation, wind, humidity, evaporation and soil moisture) and 27 fixed catchment properties (size, height, slope,

rain climate, land use, soil, geology, dams and population). Properties calculated from the flow itself were left out, so the model cannot learn the answer from its inputs.

Model. CudaLSTM with 128 hidden units and the NSE loss used for multi-basin training, 30 epochs. Training 1981–2004, validation 2005–2009, test 2010–2020. It was trained on a free Google Colab T4 GPU (run time in Section 10).

Fair comparison. Single-basin models with the old settings were trained on the *same* data and years for basin 3002 and 10 random basins, so the only difference is one model for all basins versus one model per basin.

Table 16: Test scores, 2010–2020 (medians over basins)

	NSE	**KGE**	**NSE > 0**	**NSE > 0.5**
Multi-basin model, all 202 basins	0.39	0.50	75%	42%
Multi-basin model, the 11 comparison basins	**0.32**	**0.30**	64%	**45%**
Single-basin models, the same 11 basins	0.21	0.24	73%	27%

Table 17: Test NSE and KGE in the 11 comparison basins

Basin	**NSE single**	**NSE multi**	**KGE single**	**KGE multi**
3002	0.29	**0.58**	0.34	**0.65**
3009	−1.89	**−1.54**	−1.89	**−1.72**
3073	0.76	**0.79**	0.81	**0.84**
3082	**0.71**	0.71	0.68	**0.76**
3089	0.46	**0.63**	0.52	**0.70**
4016	**0.06**	−0.05	0.24	**0.30**
4026	**0.84**	0.83	0.91	**0.91**
9001	0.21	**0.32**	0.10	**0.17**
9012	**−781**	−1185	**−103**	−117
11007	−5.19	**−1.29**	−1.18	**0.01**
17006	0.02	**0.06**	−0.48	−0.30

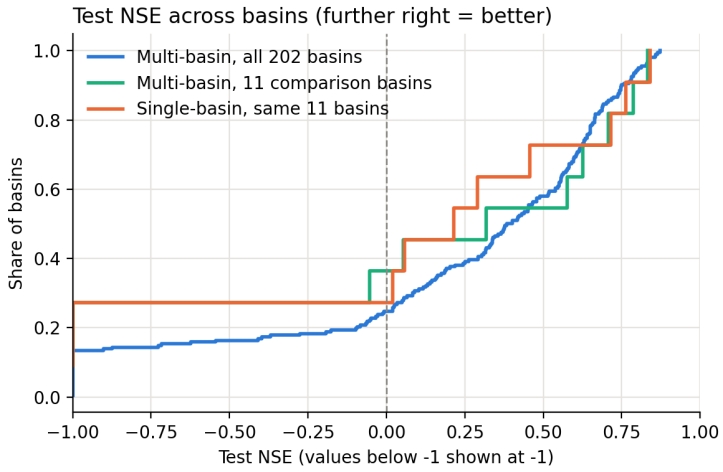


Figure 12: Share of basins with a test NSE up to a given value (further right is better; values below −1 are shown at −1).

Table 18: Multi-basin test scores by climate and dams (medians)

Group	Basins	NSE	KGE	NSE > 0
Mean rainfall: driest third	68	−0.02	0.00	47%
Mean rainfall: middle third	68	0.45	0.51	82%
Mean rainfall: wettest third	66	**0.61**	**0.66**	97%
No dams	31	0.42	0.49	71%
At least one dam	171	0.38	0.50	76%

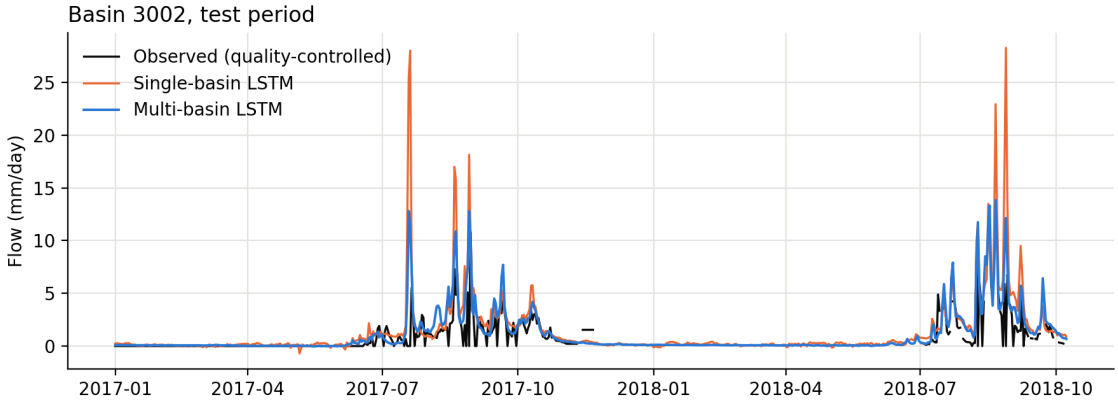


Figure 13: Basin 3002 in 2017–2018: observed flow (after cleaning) and the single- and multi-basin LSTMs trained on 1981–2004.

1. **One model for all basins beats one model per basin.** On the same data and years, the multi-basin model had a better NSE in 7 of the 11 basins and a better KGE in 10 of them, and reached NSE > 0.5 in 45% of them against 27%. For basin 3002 over 2010–2020, NSE rose from 0.29 to 0.58.

2. **Over all 202 basins the median NSE was 0.39 and the median KGE 0.50.** The CAMELS-IND authors report a median NSE of 0.59 over 159 basins (Mangukiya et al., 2025), with a different choice of basins, periods and tuning, so the numbers are not directly comparable.

3. **Climate decides how well the model works, not dams.** In the wettest third of basins the median NSE was 0.61; in the driest third it was about 0. In dry basins the river is dry most of the year and flows only after a few storms, so small errors in those events decide the score. Basins with and without dams scored about the same.

4. **Basin 3002 in 2017–2018.** Over this short window, the new models (trained on 1981–2004) did worse than Experiment 3C (trained on 2012–2015): NSE −0.38 against −0.19. Both new models overestimate the monsoon peaks while the observed flow keeps falling to zero (Figure 13). The basin may have changed over time, so recent training years fit it better; this was not tested.

5. **A few basins have extremely negative NSE** (basin 9012: −1185). When the observed flow hardly varies, NSE turns even small errors into huge negative values, so medians are reported instead of means.

Limitations. One training run without tuning (an average of several runs usually scores higher), simple cleaning rules, and a comparison with single-basin models in 11 basins only.

9 Flood Prediction with WRF-Hydro

WRF-Hydro is NCAR's hydrology model. It takes rain and weather as input, moves the water through the soil, over the land and along the river channels, and gives the river flow (m^3/s) at each point of the river. I first ran NCAR's ready-made test case (Croton river, New York) to check my build; it ran correctly and gave KGE 0.60 at two of its three gauges.

River and setup. I chose the **Kallada river at Pattazhy** (CAMELS-IND gauge 15022, 1210 km^2, one dam at Thenmala). It lies inside my WRF 3 km box, has almost a full record for August 2018, and peaked at 937 m^3/s on 16 August, the same day as the Idukki flood. I built every input myself:

- a 90 × 90 km land grid at 1 km with WPS `geogrid`;
- a 250 m river grid from the Copernicus 30 m DEM with NCAR's open-source GIS tool. The first catchment came out 16% too small (1029 km^2), because part of the upper basin drained to another river in the DEM. I fixed it by adding walls along the official CAMELS-IND outline, after which it was 1224 km^2 against 1226 km^2;
- hourly weather from ERA5 for May–August 2018 (May is only spin-up and is not scored). For a second run I replaced the rain with IMD gauge-based daily rain, spread over the hours using ERA5's hourly pattern.

I used gridded routing, a simple groundwater bucket and NCAR's default parameters, with no calibration. Each four-month run took about 30 minutes on 4 cores of my laptop.

Table 19: WRF-Hydro daily flow at Pattazhy (Kallada) against observed CAMELS-IND flow, June–August 2018

	ERA5 rain	**IMD rain**
Rain over the domain, Jun–Aug / 14–18 Aug (mm)	881 / 215	991 / 236
Correlation	0.70	**0.88**
NSE	0.04	**0.38**
KGE	0.36	**0.50**
Bias of total flow	−52%	−49%
Flood peak (observed 937 m^3/s)	**919** (−2%)	1088 (+16%)
Flood water, 14–18 Aug (model / observed)	0.64	**0.85**

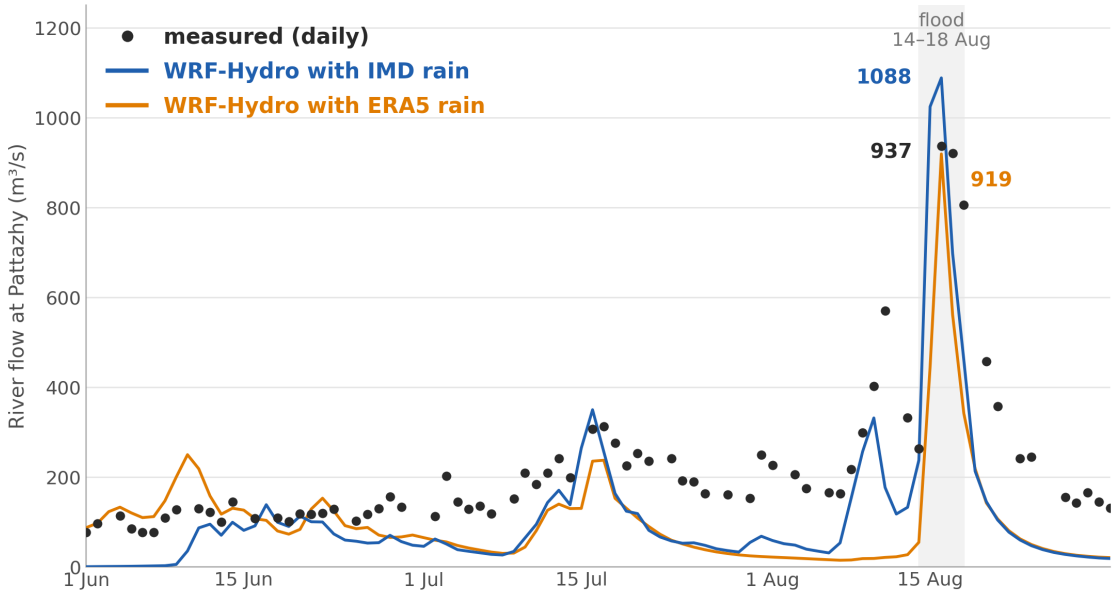


Figure 14: Daily river flow at Pattazhy (Kallada), June–August 2018: observed (black dots) against WRF-Hydro with IMD rain (blue) and with ERA5 rain (orange). The grey band marks the main flood, 14–18 August.

What I found:

1. **The flood was reproduced.** With both rain inputs the model rose on the right days for every storm of the season, and the 16 August peak was within 2% (ERA5) and 16% (IMD) of the observed 937 m^3/s.

2. **Better rain mainly improved the timing.** IMD rain raised the correlation from 0.70 to 0.88 and the NSE from 0.04 to 0.38. It caught the rises of 1 and 9–11 August that ERA5 missed, and removed a false flood in late May, when ERA5 had twice as much rain as IMD (400 against 180 mm).

3. **Between storms the model has about half the observed flow**, whichever rain is used. So this error comes from the model, not the rain: the untuned groundwater drains too fast, and the Thenmala dam, which releases water all through the monsoon, is not in the model.

4. **Gauge 15007** (no dams) followed the June and July rises, but its flood could not be checked because its observations for 15–18 August are missing.

10 Computational Performance

In this section I compare how fast each model runs on three machines: my laptop, the IIT Kanpur CSE server and a free Google Colab GPU (Table 20).

Table 20: Machines used

	Laptop	CSE server (csewsrm10)	Google Colab
CPU	Intel Core Ultra 9 185H	Intel Core i9-14900	Intel Xeon, 2 virtual CPUs
Cores	6 fast + 10 slower cores	8 fast + 16 slower cores	–
Memory	24 GB (in WSL2)	62 GB	about 12 GB
GPU	none usable	none	NVIDIA T4
MPI	OpenMPI	MPICH 5.0.2 (built by me)	–

Both CPUs mix **fast P-cores** with **slower E-cores**. As the results show, this mattered more than the number of cores.

10.1 Summary for All Models

Table 21: Run time of every model on each machine (lower is better)

Model	**Task**	**Laptop CPU**	**Server (8 P-cores)**	**Colab T4 GPU**
FourCastNet	72-hour global forecast	–	–	**41 s**
Pangu-Weather	72-hour global forecast	12.4 min	8.2 min	**1.5 min**
WRF	36-hour Mumbai forecast at 3 km	2 h 43 min	**1 h 33 min**	–
NeuralHydrology	1 epoch, basin 3002	2.0 s	1.0 s	**0.5 s**
NeuralHydrology	1 epoch, 10 basins	271.7 s	64.9 s	**13.4 s**
NeuralHydrology	full 202-basin model (30 epochs)	–	–	169 min
LSTM with DDP	1 epoch, best setting	26.1 s	**14.4 s**	–

– = not run on this machine. WRF has no GPU version here, and FourCastNet was run only on the GPU.

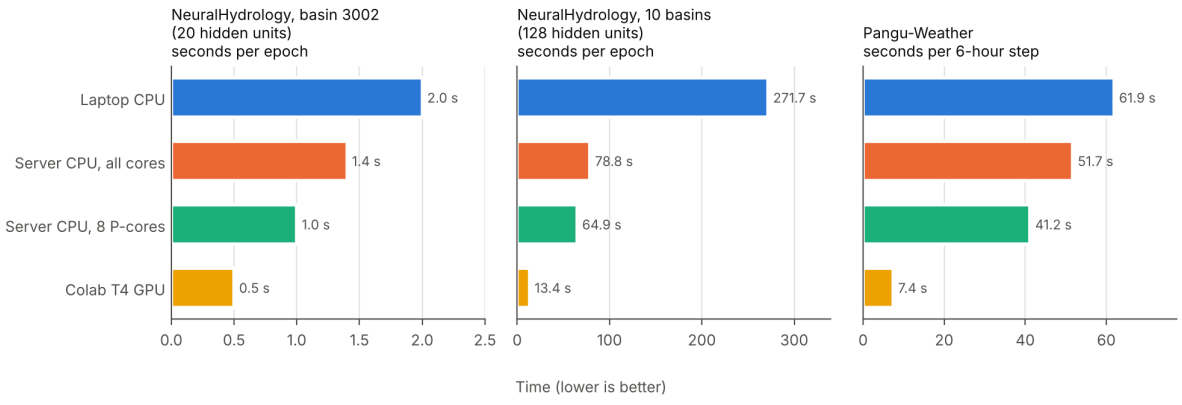


Figure 15: NeuralHydrology and Pangu-Weather on the laptop, the server (all cores and 8 P-cores) and the Colab GPU.

10.2 FourCastNet and Pangu-Weather

Both AI models ran on the Colab T4 GPU. FourCastNet made a 72-hour global forecast in 41 s. Pangu-Weather took 7.4 s per 6-hour step, so about 1.5 minutes for 72 hours, twice as slow as FourCastNet.

To time Pangu-Weather on the CPUs, I ran its real 6-hour model on a random input of the real size. One step took 61.9 s on the laptop, 51.7 s on all server cores and 41.2 s on the server's 8 P-cores. Earlier, on Colab's small CPU, it took about 4.3 minutes per step. So these models can run on a CPU, but a GPU makes them 5–8 times faster.

10.3 WRF

On the laptop. WPS (preparing the input) took about 1 minute and `real.exe` a few seconds. Almost all the time was spent in `wrf.exe` (Table 22). Each run used about 9 GB of memory.

Table 22: WRF run times on the laptop (16 processes)

	Run A (60 h)	Run B (36 h)	Run C (60 h)
`wrf.exe` time	4 h 25 min	2 h 43 min	3 h 47 min
Faster than real time	13.6×	13.3×	15.9×

On the server. I built everything WRF needs in my home folder: `tcsh`, zlib, HDF5, NetCDF, MPICH 5.0.2 and WRF 4.8.0.

Which cores to use. I ran the same 6-hour forecast on different sets of server cores (Table 23, Figure 16).

Table 23: WRF time for a 6-hour forecast on the CSE server

Processes	Cores used	Time (s)
4	4 P-cores	1386
8	8 P-cores	**947**
16	8 P-cores, 2 threads each (hyper-threading)	1013
16	16 E-cores	1734
24	8 P + 16 E-cores	1514
8	chosen by Linux	1002
16	chosen by Linux	1794
24	chosen by Linux	1473
32	chosen by Linux	1275

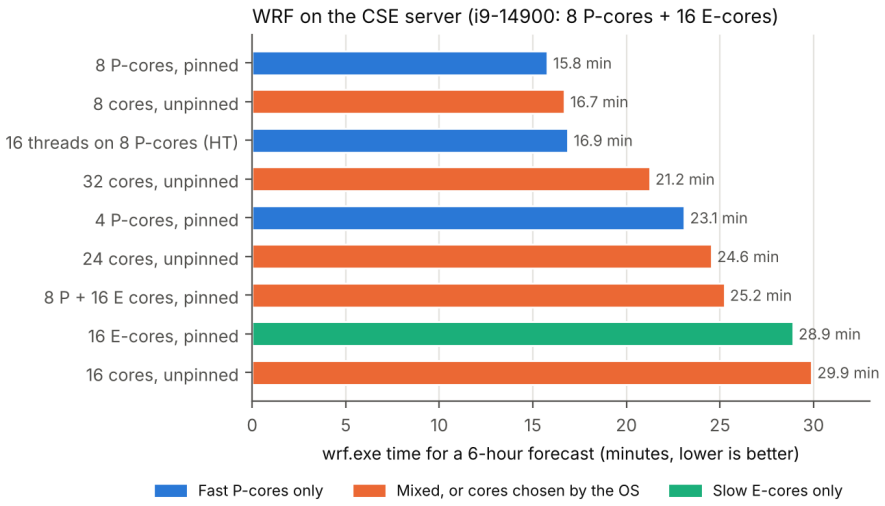


Figure 16: WRF time for the same 6-hour forecast on different sets of server cores.

- **8 P-cores were the fastest** (947 s).
- **Adding E-cores made it slower.** In WRF, all processes wait for the slowest one after every time step, so slow cores hold back the fast ones. 8 P-cores plus 16 E-cores took 60% longer than the 8 P-cores alone.
- **Hyper-threading did not help** (1013 s against 947 s).

Laptop against server. A full 36-hour forecast took 2 h 43 min on the laptop. On the server it took about 3 h 34 min when Linux chose 16 cores, but only **1 h 33 min** on the 8 P-cores, 1.8 times faster than the laptop. The 102-hour Kerala run took 4 h 50 min on the same 8 P-cores. So on these CPUs, choosing the right cores matters more than using more cores.

10.4 NeuralHydrology

I timed NeuralHydrology training on the real CAMELS-IND data for a small model (basin 3002 alone) and a large model (10 basins). Table 21 and Figure 15 give the results.

- **The GPU was the fastest**: 4 times faster than the laptop for the small model and 20 times for the large one. The bigger the model, the bigger the gain.
- **The server was 2–4 times faster than the laptop**, and again faster on its 8 P-cores than on all cores (all cores were 21–40% slower).
- **The full 202-basin model** of Section 8.2 took 169 minutes on the GPU; each single-basin comparison model took only 0.8–1.6 minutes.

10.5 Distributed PyTorch (DDP)

With DDP, several copies of a model (workers) each train on part of every batch and then share what they learned, so they all stay the same. Ran DDP on the CPU, with an LSTM of the size used for multi-basin river models and made-up river data. I ran two tests:

- **More workers, one core each** (Table 24).
- **The same threads shared by more workers** (Table 25).

Table 24: DDP training time per epoch, one core per worker (speed-up in brackets)

Workers	Laptop	Server
1	187.8 s (1.0)	120.7 s (1.0)
2	92.3 s (2.0)	63.1 s (1.9)
4	44.1 s (4.3)	32.2 s (3.7)
8	32.1 s (5.8)	17.9 s (6.7)
16	26.1 s (7.2)	18.5 s (6.5)
24	–	**14.4 s (8.4)**
32	–	15.6 s (7.7)

Table 25: DDP training time per epoch, the same threads shared by all workers

Workers	Laptop (16 threads)	Server (32 threads)
1	154.3 s	112.7 s
4	125.8 s	90.6 s
16	99.8 s	68.9 s
32	–	**57.6 s**

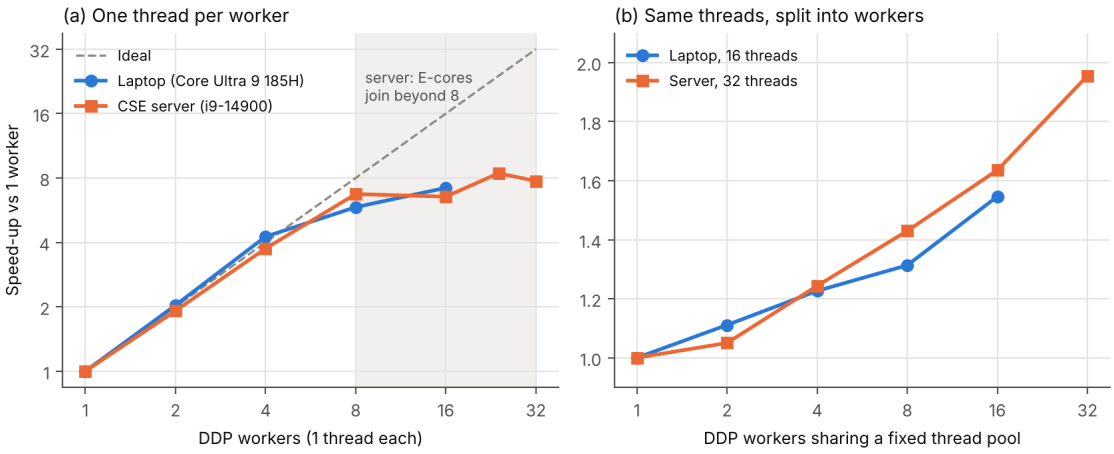


Figure 17: DDP speed-up. (a) One core per worker. (b) The same threads shared by more workers.

- **Same result, faster.** The model's accuracy was the same for any number of workers.
- **Speed grew well up to the 8 fast cores** (6.7 times faster with 8 workers). After that, extra workers ran on slow cores and it stopped improving. The best was 8.4 times faster.
- **Many small workers beat one big worker.** With the same 32 threads, 32 workers were about twice as fast as one worker.

10.6 WRF-Hydro on Eight Lab Machines

To test whether more cores make a model faster, I ran WRF-Hydro on eight identical CSE lab machines (172.27.24.10–17). They share my home folder over the network, so one build of WRF-Hydro with my MPICH 5.0.2 works on all of them. A hostfile lists the machines with 16 processes each, and `mpirun -np N -f hostfile ./wrf_hydro.exe` starts the same run on N processes. I timed three sizes of the same kind of problem (Table 26).

Table 26: The three WRF-Hydro problems used for the scaling test

Problem	Area	River grid	Simulated time
Kallada, 250 m	90 × 90 km	360 × 360 (0.13 million cells)	5 days
Kallada, 100 m	90 × 90 km	900 × 900 (0.8 million cells)	5 days
All of Kerala, 100 m	310 × 520 km	3100 × 5200 (16 million cells)	12 hours

Table 27: WRF-Hydro run time (s) on the lab machines; speed-up against 1 process in brackets

Processes	Machines	Kallada 250 m	Kallada 100 m	All Kerala 100 m
1	1	104 (1.0)	591 (1.0)	5314 (1.0)
2	1	63 (1.7)	420 (1.4)	3527 (1.5)
4	1	44 (2.4)	232 (2.6)	2268 (2.3)
8	1	33 (3.2)	202 (2.9)	1370 (3.9)
16	1	**32 (3.3)**	**193 (3.1)**	1153 (4.6)
32	2	257 (0.4)	450 (1.3)	751 (7.1)
64	4	265 (0.4)	415 (1.4)	467 (11.4)
128	8	393 (0.3)	548 (1.1)	**426 (12.5)**

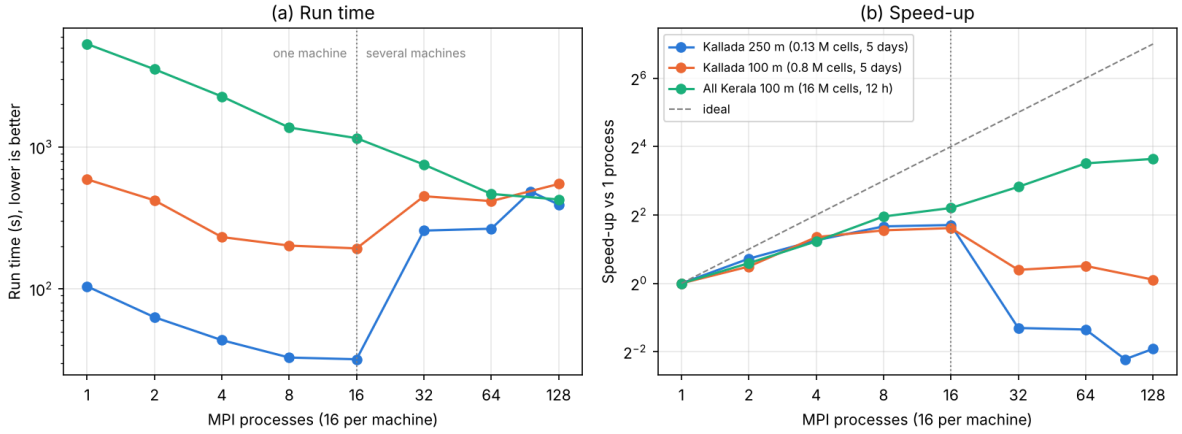


Figure 18: WRF-Hydro run time (a) and speed-up (b) for the three problems. Left of the dotted line all processes are on one machine; to the right they are spread over 2–8 machines.

Why more machines did not always help. I measured the network with a small MPI ping-pong program (Table 28). A short message between two machines takes about 100 µs, 200 times longer than inside one machine, because the lab machines are joined by ordinary 1 Gbit/s Ethernet. WRF-Hydro sends many small messages at every 10-second routing step, so this delay is paid tens of thousands of times in a run.

Table 28: MPI ping-pong between two processes

	Same machine	Two machines
Time for an 8-byte message	0.5 µs	108 µs
Speed for large messages	18,000 MB/s	116 MB/s

What I found:

1. **On one machine, more cores always made the run faster**, by 3.1–4.6 times with 16 processes.

2. **For the small Kallada problems, extra machines made it slower** (up to 8 times), because each process had little work and spent most of its time waiting for the network. A 6 times bigger grid reduced this penalty from 8 times to 2.3 times.

3. **For all of Kerala, the time fell at every step up to 128 processes**: 12.5 times faster than one process, and 8 machines were 2.7 times faster than one machine. The gain from 64 to 128 processes was small, as each process's piece became small again.

4. Adding machines pays off only when each process has much more calculation than communication; on this network that needs a problem of several million cells.

10.7 Main Points

1. **A GPU is much faster** for the AI and LSTM models: 4–20 times faster than the laptop and 2–5.5 times faster than the server.

2. **Without a GPU, the server is the better machine**: 1.5–4 times faster than the laptop, but only when the work runs on its 8 fast cores.

3. **Using all cores was always slower than using only the fast cores**, for WRF, NeuralHydrology and Pangu-Weather alike.

4. **DDP speeds up training on a CPU** (up to 8.4 times) without changing the result, and the same code would work on several GPUs or servers.

5. **More machines help only for big problems.** WRF-Hydro over all of Kerala ran 12.5 times faster on 128 processes (8 machines), but for a single small river extra machines made it slower, because of the slow network between the lab machines.

11 Conclusion

This project followed five Indian floods from the rain to the river. I tested two AI weather models (FourCastNet and Pangu-Weather), a physics weather model at high resolution (WRF), an AI river-flow model (LSTM) and a physics river-flow model (WRF-Hydro), and measured how fast each one runs on a laptop, a server, a GPU and a small cluster. The main findings are:

1. **AI weather models are fast and easy to run.** With free tools and a free Colab T4 GPU, FourCastNet made a 72-hour global forecast in 41 s and Pangu-Weather in 1.5 min. This supports their main claim of speed.

2. **FourCastNet gets the timing of flood rain right but not the amount.** It predicted 85–98% too little rain for four floods and 49% too little for Idukki. Most of this error comes from its training data: ERA5 already had 31–85% less rain than the IMD grid, and FourCastNet stayed close to ERA5. A model cannot learn heavier rain than the data it was trained on.

3. **Pangu-Weather's warning signs did not show how big a flood would be.** Its pressure and humidity fields looked realistic, but the signals were not related to flood size, and the biggest floods had some of the weakest humidity signals.

4. **A finer grid gives realistic rain peaks, but not always in the right place.** WRF at 3 km produced peaks above 500 mm for Mumbai 2005, the right size, but on the Western Ghats instead of the city. The cumulus scheme on the 9 km grid caused much of this: without it the location error fell from 221 to 84 km. For Kerala 2018, where mountains drive the rain, WRF got the regional rain within 18% of IMD.

5. **River flow can be predicted, and better rain helps the most.** One LSTM trained on 202 CAMELS-IND basins reached a median NSE of 0.39 and beat single-basin models (0.32 against 0.21 in the same basins); it works well in wet basins and poorly in dry ones. WRF-Hydro, with no calibration, caught the 2018 Kallada flood peak within 2–16%, and switching from ERA5 to IMD rain raised its NSE from 0.04 to 0.38.

6. **The hardware matters as much as the code.** A GPU was 4–20 times faster than the laptop. On the server, running only on the 8 fast cores made WRF 1.8 times faster than the laptop, and using all cores was always slower. DDP made LSTM training up to 8.4 times faster, and WRF-Hydro over Kerala ran 12.5 times faster on 128 processes across 8 machines, though extra machines did not help for a single small river.

What this means. All parts of the project point the same way. Global, coarse data describe the large-scale monsoon well, but the amount and place of heavy rain at the local scale is still the weak link. This limits the AI weather models, and it limits the river models too: the same river model improved a lot just by being given better rain. So today, AI weather models are a fast first look at a coming flood, but they should not be used alone to judge how much rain will fall.

Future work.

- Fine-tune an AI weather model on better rain data, such as the IMD grid or satellite rain, and test whether it predicts more of the flood rain.
- Join the chain end to end: use an AI forecast to drive WRF at 3 km, and feed WRF's rain into WRF-Hydro to forecast the river days ahead.
- Calibrate WRF-Hydro, add the Thenmala dam, and test more rivers and floods.
- Run the AI models directly alongside a physics model on the same machine, so their forecasts can be used while the simulation is still running.

Code and Implementation

All code, configuration files and plotting scripts used in this project are available on GitHub:

`https://github.com/Arijit030701/Flood-Forecasting-using-AI-LSTM-and-Simulation`

The repository is organised by part of the project: `1_ai_models` (FourCastNet and Pangu-Weather runs and analysis), `2_wrf` (WRF setup and runs), `3_performance` (timing on the laptop, server and GPU), `4_neuralhydrology` (LSTM experiments), `5_wrf_hydro` (WRF-Hydro setup for the Kallada river) and `6_cluster_scaling` (WRF-Hydro on eight lab machines).

References

1. Bi, K., Xie, L., Zhang, H., Chen, X., Gu, X., & Tian, Q. (2023). Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 619, 533–538.

2. Pathak, J., et al. (2022). FourCastNet: A Global Data-driven High-resolution Weather Model using Adaptive Fourier Neural Operators. *arXiv:2202.11214*.

3. Hersbach, H., et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049.

4. Pai, D. S., Sridhar, L., Rajeevan, M., Sreejith, O. P., Satbhai, N. S., & Mukhopadhyay, B. (2014). Development of a new high spatial resolution ($0.25^\circ \times 0.25^\circ$) long period (1901–2010) daily gridded rainfall data set over India and its comparison with existing data sets over the region. *MAUSAM*, 65(1), 1–18.

5. India Meteorological Department (IMD) – station rainfall records for Santacruz (Mumbai) and Safdarjung (Delhi), and event reports for Chennai and Idukki.

6. ECMWF `ai-models` toolkit and the `ai-models-panguweather` / `ai-models-fourcastnet` plugins (open-source, GitHub).

7. `imdlib`: Python package for reading IMD gridded data (open-source, PyPI).

8. Kratzert, F., Gauch, M., Nearing, G., & Klotz, D. (2022). NeuralHydrology – A Python library for Deep Learning research in hydrology. *Journal of Open Source Software*, 7(71), 4050.

9. Kratzert, F., Klotz, D., Brenner, C., Schulz, K., & Herrnegger, M. (2018). Rainfall–runoff modelling using Long Short-Term Memory (LSTM) networks. *Hydrology and Earth System Sciences*, 22, 6005–6022.

10. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.

11. Mangukiya, N. K., Kumar, K. B., Dey, P., Sharma, S., Bejagam, V., Mujumdar, P. P., & Sharma, A. (2025). CAMELS-IND: hydrometeorological time series and catchment attributes for 228 catchments in Peninsular India. *Earth System Science Data*, 17, 461–491.

12. Newman, A. J., et al. (2015). Development of a large-sample watershed-scale hydrometeorological data set for the contiguous USA. *Hydrology and Earth System Sciences*, 19, 209–223.

13. Addor, N., Newman, A. J., Mizukami, N., & Clark, M. P. (2017). The CAMELS data set: catchment attributes and meteorology for large-sample studies. *Hydrology and Earth System Sciences*, 21, 5293–5313.

14. National Centers for Environmental Prediction (2000). NCEP FNL Operational Model Global Tropospheric Analyses, continuing from July 1999. NSF NCAR Research Data Archive, `https://doi.org/10.5065/D6M043C6`.

15. Nash, J. E., & Sutcliffe, J. V. (1970). River flow forecasting through conceptual models part I – A discussion of principles. *Journal of Hydrology*, 10(3), 282–290.

16. Gupta, H. V., Kling, H., Yilmaz, K. K., & Martinez, G. F. (2009). Decomposition of the mean squared error and NSE performance criteria. *Journal of Hydrology*, 377, 80–91.

17. Knoben, W. J. M., Freer, J. E., & Woods, R. A. (2019). Technical note: Inherent benchmark or not? Comparing Nash–Sutcliffe and Kling–Gupta efficiency scores. *Hydrology and Earth System Sciences*, 23, 4323–4331.

18. Skamarock, W. C., et al. (2019). A Description of the Advanced Research WRF Model Version 4. NCAR Technical Note NCAR/TN-556+STR.

19. Gochis, D. J., et al. (2020). The WRF-Hydro Modeling System Technical Description (Version 5.1.1). NCAR Technical Note.

20. Hersbach, H., et al. (2023). ERA5 hourly data on single levels and on pressure levels from 1940 to present. Copernicus Climate Change Service (C3S) Climate Data Store (CDS), `https://doi.org/10.24381/cds.adbb2d47` and `https://doi.org/10.24381/cds.bd0915c6`.

21. Guibas, J., Mardani, M., Li, Z., Tao, A., Anandkumar, A., & Catanzaro, B. (2022). Adaptive Fourier Neural Operators: Efficient token mixers for transformers. *International Conference on Learning Representations (ICLR)*.

22. Kratzert, F., Klotz, D., Shalev, G., Klambauer, G., Hochreiter, S., & Nearing, G. (2019). Towards learning universal, regional, and local hydrological behaviors via machine learning applied to large-sample datasets. *Hydrology and Earth System Sciences*, 23, 5089–5110.

23. Hong, S.-Y., & Lim, J.-O. J. (2006). The WRF single-moment 6-class microphysics scheme (WSM6). *Journal of the Korean Meteorological Society*, 42(2), 129–151.

24. Thompson, G., Field, P. R., Rasmussen, R. M., & Hall, W. D. (2008). Explicit forecasts of winter precipitation using an improved bulk microphysics scheme. Part II: Implementation of a new snow parameterization. *Monthly Weather Review*, 136, 5095–5115.

25. Kain, J. S. (2004). The Kain–Fritsch convective parameterization: An update. *Journal of Applied Meteorology*, 43, 170–181.

26. Zhang, C., & Wang, Y. (2017). Projected future changes of tropical cyclone activity over the western North and South Pacific in a 20-km-mesh regional climate model. *Journal of Climate*, 30, 5923–5941. (New Tiedtke cumulus scheme.)

27. von Storch, H., Langenberg, H., & Feser, F. (2000). A spectral nudging technique for dynamical downscaling purposes. *Monthly Weather Review*, 128, 3664–3673.

28. European Space Agency (2021). Copernicus DEM: Global and European Digital Elevation Model (GLO-30). Copernicus Data Space Ecosystem.

29. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

30. Li, S., et al. (2020). PyTorch Distributed: Experiences on accelerating data parallel training. *Proceedings of the VLDB Endowment*, 13(12), 3005–3018.

31. Gropp, W., Lusk, E., Doss, N., & Skjellum, A. (1996). A high-performance, portable implementation of the MPI message passing interface standard. *Parallel Computing*, 22(6), 789–828.

32. ECMWF `ai-models`: `https://github.com/ecmwf-lab/ai-models`.

33. `imdlib`: `https://github.com/iamsaswata/imdlib`.